

Taller — Complejidad algorítmica y Arreglos

SD1004 · Estructura de Datos — Institución Universitaria Pascual Bravo Docente:
Juan Duque · Semestre 2026-II Cubre: Clase 2 (Big O) · Clase 3 (Arreglos 1D) · Clase 4 (Arreglos 2D)

Instrucciones generales:

- Responde directamente en este documento (edítalo en Word, Markdown o entrégalo en PDF).
- No necesitas escribir código en ningún lenguaje: usa pseudocódigo, dibujos o explicaciones en tus propias palabras. Lo que se evalúa es el razonamiento, no la sintaxis.
- Donde se pida un dibujo (arreglos, memoria), puedes hacerlo a mano y adjuntar una foto, o usar draw.io.
- Justifica siempre tu respuesta — una respuesta correcta sin justificación no cuenta completa.
- Fecha y forma de entrega: la indicada en el aula/repositorio del curso.

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

Para cada situación, indica si corresponde a $O(1)$, $O(\log n)$, $O(n)$ o $O(n^2)$, y explica por qué usando tus propias palabras (no hace falta fórmula matemática).

- Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.

R/= $O(\log n)$ Porque cuando abrimos el diccionario por la mitad en cada paso, descartamos la mitad de las opciones

- Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.

R/= $O(n)$. Tienes que hacer una operación por cada elemento

- Sacar la primera carta de un mazo ya barajado.

R/= $O(1)$. Sacar la primera carta toma exactamente el mismo tiempo sin importar si el mazo tiene 52 o si tiene muchas más cartas.

- Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.

R/= $O(n^2)$ Para cada uno de los n estudiantes, se debe recorrer otra vez a los otros n estudiantes, es un proceso de parejas ($n \times n$).

- Consultar la hora en tu reloj.

R/= $O(1)$ Solo es una acción directa porque se mira el reloj una vez y no depende de ninguna cantidad de datos

1.2 De la vida real al análisis

Piensa en una tarea cotidiana (que no sea de las ya usadas en clase) que hagas de forma $O(n)$ — es decir, que si la cantidad de elementos se duplica, el tiempo que tardas también se duplica.

- Descríbela en 2-3 líneas.
- Explica qué pasaría si la "entrada" (n) se hiciera 10 veces más grande.

R/= **Tarea $O(n)$** : Lavar los platos a mano.

- **¿Qué pasaría si la entrada (n) se hace 10 veces más grande?** Si normalmente lavo 5 platos y me demoro 5 minutos, lavar 50 platos (10 veces más) me tomará 50 minutos (10 veces más tiempo). El esfuerzo crece de forma directamente proporcional y lineal a la cantidad de platos.

1.3 ¿Cuál escalar mejor?

Tienes dos formas de resolver el mismo problema:

- Forma A: $O(n^2)$
- Forma B: $O(n \log n)$

Para un conjunto pequeño de datos ($n = 5$), la Forma A es más rápida en la práctica. Aun así, ¿cuál recomendarías usar si no sabes qué tan grande será n en el futuro? Justifica tu respuesta pensando en qué pasa a medida que n crece.

R/= **Forma B** ($O(n \log n)$). Aunque $O(n^2)$ sea más rápida con 5 elementos (quizás porque su código base es más sencillo o tiene menos operaciones previas), a medida que n crezca a cientos o miles, el tiempo de $O(n^2)$ "explotará" rápidamente (1,000 elementos = 1,000,000 de operaciones). En cambio, $O(n \log n)$ crecerá de manera muchísimo más suave y controlada, asegurando que el programa no se congele cuando los datos aumenten en el futuro.

1.4 Verdadero o falso (justifica siempre)

a. Un algoritmo $O(1)$ siempre es más rápido en segundos reales que uno $O(n)$.

R/= **Falso**: Big O no mide segundos, mide escalabilidad (cuántas operaciones extra se hacen al crecer n). Un algoritmo $O(1)$ puede tener un tiempo base altísimo (ej. toma 5

minutos fijos ejecutarlo), mientras que un algoritmo $O(n)$ para un arreglo de 2 elementos puede resolverse en milisegundos.

b. La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

R/= **Verdadero**: La notación Big O analiza la tendencia y la tasa de crecimiento de las operaciones a medida que la entrada (n) tiende al infinito, abstrayéndose del hardware, el lenguaje o los segundos reales.

c. Buscar por índice en un arreglo (arreglo[5]) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

R/= **Falso**: Buscar por índice en un arreglo es $O(1)$. La computadora no recorre los elementos uno por uno; usa una fórmula matemática para calcular instantáneamente la dirección exacta en la memoria RAM y salta directo a ella.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Vas a guardar las calificaciones de 6 estudiantes de un curso en un arreglo llamado notas.

- Dibuja el arreglo con sus 6 casillas, mostrando el índice de cada una (recuerda: empieza en 0).

VALOR	3,5	4,2	2,8	5,0	3,9	4,5
INDICE	[0]	[1]	[2]	[3]	[4]	[5]

- Si notas = [3.5, 4.2, 2.8, 5.0, 3.9, 4.5], ¿qué valor y qué índice tiene la tercera nota que ingresaste?

R/= Tiene el valor de 2.8 y se encuentra en el índice [2]

- ¿Cuál es la instrucción (en pseudocódigo) para acceder directamente a la nota del último estudiante sin recorrer el arreglo?

R/= Nota_Ultimo = notas[5]

2.2 Direcciones de memoria

Supón que el arreglo notas del punto anterior se guarda en memoria RAM empezando en la dirección 0x2000, y cada valor ocupa 4 bytes.

- Calcula la dirección de memoria de notas[0], notas[3] y notas[5], usando la fórmula:
dirección = base + (índice × tamaño)

R/=

notas[0]: $0x2000 + (0 \times 4) = 0x2000$

notas[3]: $0x2000 + (3 \times 4) = 0x2000 + 12 \text{ bytes} = 0x200C$

notas[5]: $0x2000 + (5 \times 4) = 0x2000 + 20 \text{ bytes} = 0x2014$

- Explica en tus palabras por qué acceder a notas[3] es igual de rápido que acceder a notas[0] — no importa cuál pidas, es $O(1)$.

R/= la computadora aplica la fórmula de multiplicación y suma en un solo ciclo para ir a esa dirección de memoria ram por eso es que da igual cual se pida siempre va a tardar lo mismo porque no recorre los puesto uno por uno

2.3 Búsqueda lineal vs. binaria

Tienes el arreglo ordenado edades = [15, 18, 20, 23, 27, 31, 35, 40].

- Simula paso a paso una búsqueda lineal para encontrar el valor 31. ¿Cuántas comparaciones hiciste?

R/=

compara 0 (15) == 31? No.

compara 1 (18) == 31? No.

compara 2 (20) == 31? No.

compara 3 (23) == 31? No.

compara 4 (27) == 31? No.

compara 5 (31) == 31? Sí. **(6 comparaciones)**

- Simula paso a paso una búsqueda binaria para encontrar el mismo valor 31. ¿Cuántas comparaciones hiciste?

R/=

- Cento el índice entre 0 y 7 es $(0+7)/2 = 3.5$ compara: $31 > 23$, buscamos en la mitaño derecha (índices 4 a 7)

- centro entre 4 y7 es $(4+7)/2 = 5.5$, índice 5 (31) compara: ENCONTRADO
(2 comparaciones)

- ¿Por qué la búsqueda binaria solo funciona si el arreglo ya está ordenado?

R/= Porque la estrategia de la búsqueda binaria es saber hacia qué lado descartar datos. Si el arreglo está desordenado, saber que 31 es mayor que 23 no garantiza que el 31 esté a la derecha; podría estar en cualquier parte.

- Si el arreglo tuviera 1 millón de elementos, ¿cuál de las dos búsquedas seguirías usando? Justifica con lo visto sobre $O(n)$ vs. $O(\log n)$.

R/= Seguiría usando **búsqueda binaria** ($O(\log n)$). En el peor de los casos, la lineal ($O(n)$) me haría comprobar 1,000,000 de casillas. La binaria, dividiendo entre 2 cada vez, me encontraría el dato en aproximadamente **20 comparaciones**.

2.4 Insertar un elemento

Tienes edades = [15, 18, 20, 23, 27] (5 casillas, sin espacio libre).

- ¿Qué tan costoso es insertar un nuevo valor al final, si hay espacio disponible? ¿Y si no hay espacio y hay que crear un arreglo más grande?

R/= si hay espacio es muy economico $O(1)$, solo se pone el valor en el indice que esta disponible

si no hay espacio es costoso $O(1)$, hay que crear un arreglo nuevo mas grande y copiar todos los elementos que ya existen, un por uno al nuevo arreglo

- ¿Qué tan costoso es insertar el valor 21 en la mitad (para que quede ordenado)? Explica qué hay que hacer con los demás elementos.

R/= Es costoso $O(n)$ se deben mover hacia la derecha todos los elementos mayores que 21 para abrir espacio donde irá el 21

- Compara ambos casos usando notación Big O.

R/= insertar al final con espacio libre: $O(1)$
insertar en la mitad o redimensionar arreglo: $O(n)$

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Vas a guardar la disposición de un salón de clase de 3 filas x 4 columnas, donde cada casilla indica si el puesto está ocupado (1) o libre (0).

- Dibuja la matriz salon con sus 3 filas y 4 columnas, con los índices [fila][columna] de cada casilla.

fila/columna	columna 0	columna 1	columna 2	columna 3
fila 0	Salon[0][0]	Salon[0][1]	Salon[0][2]	Salon[0][3]
fila 1	Salon[1][0]	Salon[1][1]	Salon[1][2]	Salon[1][3]
fila 2	Salon[2][0]	Salon[2][1]	Salon[2][2]	Salon[2][3]

- Escribe (en pseudocódigo) cómo accedes al puesto de la fila 2, columna 3.

R/= estado = salon[2][3]

- ¿Cuántas casillas tiene en total esta matriz? ¿Cómo lo calculas en general para filas x columnas?

R/= Esta matriz tiene: $3 \times 4 = 12$ casillas

Fórmula general: Total = filas x columnas

3.2 De 2D a memoria (row-major)

La memoria RAM es una sola fila continua de casillas — no existen "filas y columnas" físicamente en la RAM. Por eso una matriz se guarda "aplanada", fila por fila.

Supón que el salón (3 filas x 4 columnas) se guarda empezando en la dirección 0x1000, con cada valor ocupando 4 bytes, en orden por filas (row-major).

1. Dibuja cómo quedarían las 12 casillas de salón en una sola fila de memoria (aplanadas).

R/= [0,0] [0,1] [0,2] [0,3] [1,0] [1,1] [1,2] [1,3] [2,0] [2,1] [2,2] [2,3]

2. Calcula la dirección de memoria de la casilla [1][2], usando la fórmula: dirección = base + ((fila x número_de_columnas + columna) x tamaño)

R/= Fórmula: dirección = base + ((fila x columnas) + columna) x tamaño Datos: Base = 0x1000 (4096), Filas = 3, Columnas = 4, Tamaño = 4 bytes. dirección = $0x1000 + ((1 \times 4) + 2) \times 4 = 0x1000 + (4 + 2) \times 4 = 0x1000 + 24 \text{ bytes} = 0x1018$

3. Calcula también la dirección de [0][0] y de [2][3] (la última casilla).

R/= [0][0]: $0 \times 1000 + ((0 \times 4) + 0) \times 4 = 0 \times 1000$

[2][3]: $0 \times 1000 + ((2 \times 4) + 3) \times 4 = 0 \times 1000 + (8 + 3) \times 4 = 0 \times 1000 + 44 \text{ bytes} = 0 \times 102C$

3.3 Recorrido con ciclos anidados

Quieres contar cuántos puestos están ocupados (1) en toda la matriz salon.

1. Describe en pseudocódigo el recorrido usando dos ciclos anidados (uno para filas, otro para columnas).

R/= ocupados = 0

Para f desde 0 hasta 2 (filas):

Para c desde 0 hasta 3 (columnas):

Si salon[f][c] == 1:

ocupados = ocupados + 1

2. Si la matriz tiene f filas y c columnas, ¿cuál es la complejidad de este recorrido en notación Big O? Explica por qué.

R/= $O(n)$ donde n es el número total de casillas. Hay que revisar cada casilla individualmente exactamente una vez.

3. ¿Qué pasaría con el tiempo de ejecución si f y c se duplican ambos a la vez?

R/= Sí las filas se duplican (2f) y las columnas se duplican (2c), la cantidad total de casillas se multiplica por 4 ($2f \times 2c = 4fc$). Entonces, el tiempo de ejecución se multiplicará por 4.

3.4 Conectando todo

Entender la complejidad algorítmica (Big O) es lo que te permite elegir correctamente entre estructuras y algoritmos para que un programa escale bien. Los arreglos 1D y 2D son valiosísimos porque la memoria RAM los aloja de forma contigua, permitiendo que leer cualquier dato (sabiendo su índice) sea inmediato ($O(1)$). Sin embargo, buscar información a ciegas dentro de ellos es ineficiente ($O(n)$) si hay millones de datos. Aquí es donde conectamos con la lógica: si mantenemos nuestros arreglos ordenados, podemos aplicar algoritmos brillantes como la búsqueda binaria ($O(\log n)$), demostrando que elegir la estructura de datos correcta combinada con el algoritmo óptimo es el verdadero secreto para procesar datos a gran escala.